3.6 Convolutional neural networks (LeNet)

- we now have all the ingredients required to assemble a fully-functional CNN
- in this section, we will introduce *LeNet*, among the first published CNNs to capture wide attention for its performance on computer vision tasks
- the model was introduced by (and named for) Yann LeCun, then a researcher at AT&T Bell Labs, for the purpose of recognizing handwritten digits in images
- this work represented the culmination of a decade of research developing the technology
- in 1989, LeCun published the first study to successfully train CNNs via backpropagation

3.6 Convolutional neural networks (LeNet)

- at the time, LeNet achieved outstanding results, matching the performance of support vector machines, then a dominant approach in supervised learning
- LeNet was eventually adapted to recognize digits for processing deposits in ATM machines; to this day, some ATMs still run the code that Yann LeCun and his colleague Leon Bottou wrote in the 1990s
- at a high level, LeNet (LeNet-5) consists of two parts:
  - a convolutional encoder consisting of two convolutional layers
  - a dense block consisting of three fully-connected layers; the architecture is summarized in Figure 9

3.6 Convolutional neural networks (LeNet)

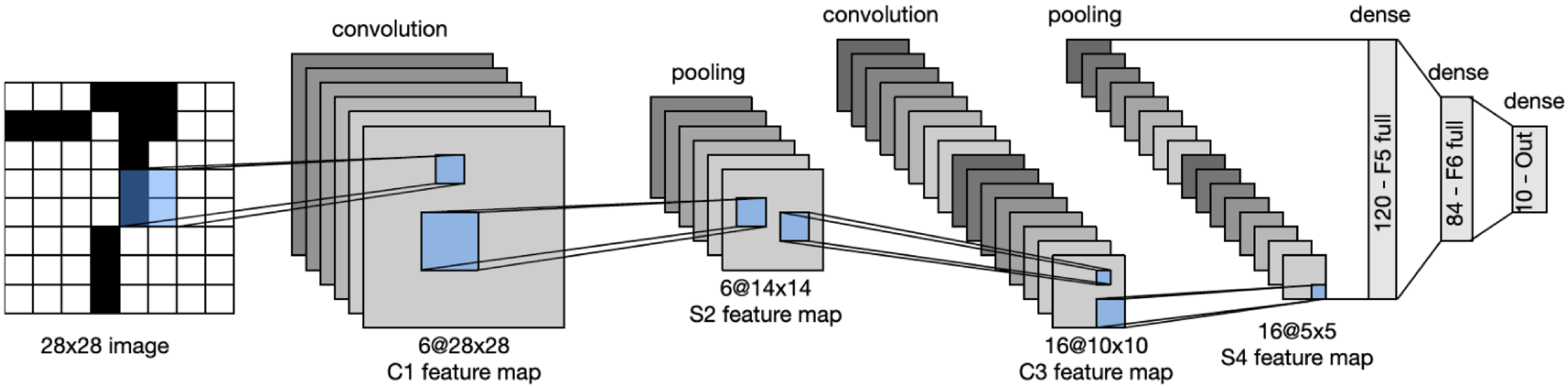

Figure 9: Data flow in LeNet. The input is a handwritten digit and the output is a probability over 10 possible outcomes.

3.6 Convolutional neural networks (LeNet)

- the basic units in each convolutional block are a convolutional layer, a sigmoid activation function, and a subsequent average pooling operation
- note that, while ReLUs and max-pooling work better, these discoveries had not yet been made in the 1990s
- each convolutional layer uses a 5×5 kernel and a sigmoid activation function
- these layers map spatially arranged inputs to a number of two-dimensional feature maps, typically increasing the number of channels
- the first convolutional layer has 6 output channels, while the second has 16
- each 2×2 pooling operation (stride 2) reduces dimensionality by a factor of 4 via spatial downsampling
- the convolutional block emits an output with shape given by (batch size, number of channels, height, width)

3.6 Convolutional neural networks (LeNet)

- in order to pass output from the convolutional block to the dense block, we must *flatten* each example in the mini-batch
- in other words, we take this four-dimensional input and transform it into the two-dimensional input expected by fully-connected layers: as a reminder, the two-dimensional representation that we desire uses the first dimension to index examples in the mini-batch and the second to give the flat vector representation of each example
- LeNet's dense block has three fully-connected layers, with 120, 84, and 10 outputs, respectively
- because we are still performing classification, the 10-dimensional output layer corresponds to the number of possible output classes
- by passing a single-channel (black and white) 28×28 image through the network, the model is depicted in Figure 10
- we took a small liberty with the original model, removing the Gaussian activation in the final layer; other than that, this network matches the original LeNet-5 architecture

3.6 Convolutional neural networks (LeNet)

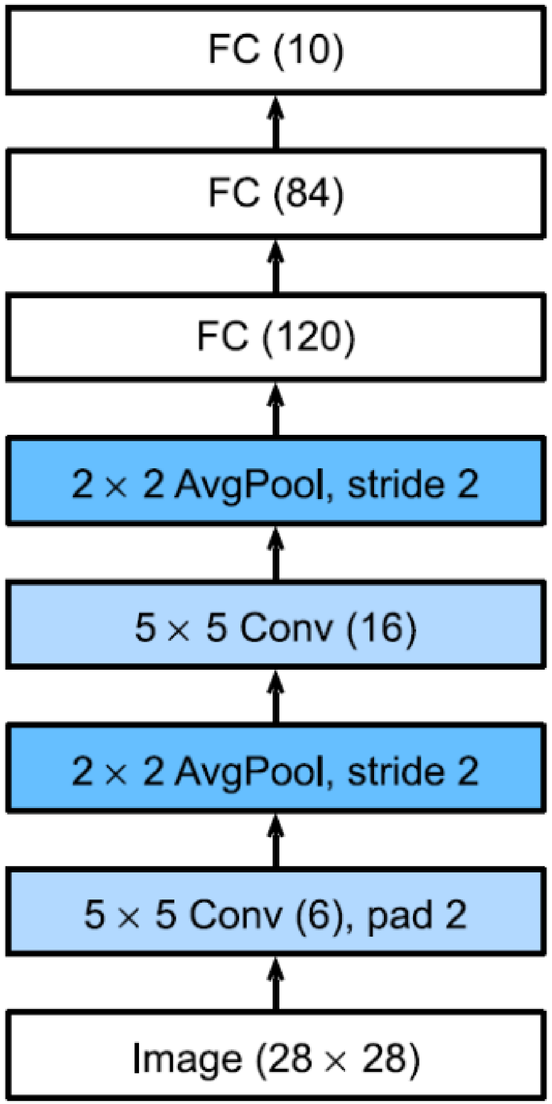

Figure 10: Compressed notation for LeNet-5.

3.6 Convolutional neural networks (LeNet)

- note that the height and width of the representation at each layer throughout the convolutional block is reduced (compared with the previous layer)
- the first convolutional layer uses 2 pixels of padding to compensate for the reduction in height and width that would otherwise result from using a 5×5 kernel
- in contrast, the second convolutional layer doesn't use padding, and thus the height and width are both reduced by 4 pixels
- as we go up the stack of layers, the number of channels increases layer-over-layer from 1 in the input to 6 after the first convolutional layer and 16 after the second convolutional layer
- however, each pooling layer halves the height and width
- finally, each fully-connected layer reduces dimensionality, finally emitting an output whose dimension matches the number of classes